

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

Projeto de Qualidade de Software

Rede Raízes do Nordeste - Pedido para retirada rápida com pagamento simulado

Curso	Análise e desenvolvimento de sistemas
Disciplina	Projeto: Engenharia de Software
Aluno	Lincoln Silveira
RU	4509553
Trilha escolhida	Qualidade de Software

Sumário

1. Introdução	4
1.1 Contexto do estudo de caso.....	4
1.2 Objetivos do projeto.....	4
1.3 Principais usuários e partes interessadas.....	4
1.4 Relevância do sistema	4
1.5 Recorte adotado	5
2. Análise e Requisitos.....	5
2.1 Escopo.....	5
2.2 Regras de negócio iniciais	5
2.3 Requisitos funcionais.....	5
2.4 Requisitos não funcionais	6
2.5 Requisitos de qualidade mensuráveis	6
2.6 Critérios de aceitação.....	7
3. Modelagem e Arquitetura.....	7
3.1 Decisão arquitetural.....	7
3.2 Visão macro da solução	7
3.3 Contextos de domínio.....	8
3.4 Estrutura do backend	8
3.5 Estrutura do frontend.....	9
3.6 Contrato mínimo da API	10
3.7 Modelo de dados mínimo	10
3.8 Estratégia de testes e ferramentas	10
4. Entrega Técnica	11
4.1 Recorte técnico entregue	11
4.2 Estrutura sugerida do repositório final	11
4.3 Como executar localmente.....	11
4.4 Quadros de evidência da entrega técnica.....	11
EV-001 - Estrutura do repositório e arquitetura modular.....	11
EV-002 - Backend em execução	14
EV-003 - Frontend em execução.....	14
EV-004 - Documentação do projeto	15
5. Plano de Testes e Evidências.....	16
5.1 Diretriz central de QA	16
5.2 Plano de Garantia da Qualidade	16
5.2.1 Plano de Qualidade	17

5.2.2 Plano de Testes	17
5.2.3 Cronograma de Validação	17
5.2.4 Papéis e Responsabilidades	18
5.3 Matriz de rastreabilidade	18
5.4 Métricas e indicadores.....	20
5.5 Evidências por tipo de teste	21
5.5.1 Cenários de teste.....	21
6. Conclusão	37
6.1 Lições aprendidas	37
6.2 Desafios e pontos de atenção.....	37
6.3 Evoluções futuras	37
7. Referências	38

1. Introdução

1.1 Contexto do estudo de caso

O estudo de caso da rede Raízes do Nordeste descreve uma rede de lanchonetes nordestinas em expansão, com diferentes unidades, canais de atendimento e regras operacionais locais. O sistema proposto precisa apoiar a experiência do cliente em canais digitais, especialmente em dispositivos móveis, garantindo padronização da jornada, confiabilidade do pedido, desempenho, tratamento de falhas, privacidade e rastreabilidade.

A realidade descrita envolve cardápio variável por unidade, produtos sazonais, pedidos para retirada rápida, pagamentos desacoplados, necessidade de auditoria e preocupação explícita com LGPD. Por isso, a solução foi conduzida pela trilha de Qualidade de Software, com foco em requisitos verificáveis, critérios de aceitação, testes, métricas e evidências.

1.2 Objetivos do projeto

Objetivo	Descrição
Objetivo geral	Planejar, documentar e validar uma estratégia de QA para uma fatia crítica do sistema: pedido para retirada rápida com pagamento simulado.
Objetivo técnico	Construir uma fatia vertical testável, com backend, frontend, API, regras de negócio, testes automatizados e evidências.
Objetivo de qualidade	Demonstrar rastreabilidade entre risco, requisito, critério de aceitação, teste, evidência e métrica.
Objetivo formativo	Aplicar visão profissional de requisitos, QA, arquitetura modular, testes e documentação em um cenário próximo de mercado.

1.3 Principais usuários e partes interessadas

Usuário/parte interessada	Interesse principal	Relevância para QA
Cliente	Fazer pedido rápido, visualizar status e receber comunicação clara.	Usabilidade, desempenho, responsividade e mensagens de erro.
Unidade/atendente	Receber pedido correto e evitar retrabalho.	Validação de disponibilidade, status e consistência.
Matriz/franqueadora	Padronizar operação e acompanhar dados consolidados.	Rastreabilidade, auditoria e métricas.
Equipe de desenvolvimento	Implementar com baixo acoplamento e manutenção clara.	Arquitetura modular e testes por domínio.
Equipe de QA	Planejar, executar e evidenciar qualidade.	Matriz, cenários, automação e relatórios.
Responsável por privacidade	Garantir uso adequado de dados pessoais.	LGPD, consentimento, minimização e logs seguros.

1.4 Relevância do sistema

O sistema é relevante porque conecta uma jornada crítica de negócio - pedido e retirada - com requisitos de qualidade diretamente ligados a faturamento, experiência do cliente, operação da loja e imagem da marca. Em horários de pico, falhas de disponibilidade, lentidão, confirmação incorreta ou produtos indisponíveis podem gerar impacto operacional imediato.

1.5 Recorte adotado

A feature escolhida para a análise, implementação parcial e evidências foi:

Pedido para retirada rápida com pagamento simulado.

Esse recorte permite validar seleção de unidade, cardápio por unidade, criação de pedido, pagamento aprovado/negado/falha, status do pedido, auditoria, LGPD e responsividade mobile, sem implementar o sistema completo de franquias.

2. Análise e Requisitos

2.1 Escopo

Dentro do escopo	Fora do escopo
Análise de riscos, requisitos e critérios de aceite.	Sistema completo de franquias em produção.
Feature de pedido para retirada rápida.	Gateway real de pagamento.
Backend, frontend e testes da fatia mínima.	ERP, fiscal, estoque real e operação comercial.
Plano de QA, métricas e evidências.	Infraestrutura física e suporte pós-implantação.
LGPD no fluxo e na documentação.	Programa completo de fidelização.

2.2 Regras de negócio iniciais

ID	Regra de negócio
RN-01	O cliente seleciona uma unidade antes de visualizar o cardápio.
RN-02	O cardápio exibe apenas produtos disponíveis na unidade selecionada.
RN-03	Um pedido válido possui ao menos um item.
RN-04	Quantidades devem ser maiores que zero.
RN-05	O pedido só é confirmado com pagamento simulado aprovado.
RN-06	Falhas de pagamento devem ser tratadas e auditáveis.
RN-07	O status do pedido deve refletir o resultado do fluxo.
RN-08	A jornada deve evitar duplicidade por repetição de ação.

2.3 Requisitos funcionais

ID	Requisito funcional	Prioridade
RF-01	Permitir seleção de unidade antes do pedido.	Alta
RF-02	Exibir cardápio da unidade selecionada.	Alta
RF-03	Permitir adicionar produtos disponíveis ao pedido.	Alta
RF-04	Permitir ajustar quantidade antes da confirmação.	Média
RF-05	Rejeitar produto indisponível para a unidade.	Alta
RF-06	Impedir pedido vazio.	Alta

ID	Requisito funcional	Prioridade
RF-07	Exibir revisão com itens, quantidades e total.	Alta
RF-08	Calcular valor total a partir dos itens válidos.	Alta
RF-09	Solicitar pagamento simulado após confirmação.	Alta
RF-10	Registrar retorno APROVADO, NEGADO ou FALHA.	Alta
RF-11	Confirmar pedido apenas com pagamento aprovado.	Alta
RF-12	Exibir mensagem clara para pagamento negado.	Alta
RF-13	Exibir mensagem clara e rastreável para falha de pagamento.	Alta
RF-14	Permitir consulta de status do pedido.	Alta
RF-15	Registrar eventos relevantes de auditoria.	Média
RF-16	Evitar duplicidade na confirmação.	Alta
RF-17	Exibir aviso de privacidade quando houver uso de dados pessoais.	Alta
RF-18	Registrar consentimento quando aplicável.	Alta

2.4 Requisitos não funcionais

ID	Dimensão	Meta inicial
RNF-01	Desempenho do cardápio	P95 até 2 segundos.
RNF-02	Desempenho da criação de pedido	P95 até 2 segundos.
RNF-03	Disponibilidade	99,5% mensal como referência.
RNF-04	Usabilidade	Até 3 etapas principais após a escolha dos produtos.
RNF-05	Responsividade	Fluxo válido em 360px
RNF-06	Confiabilidade	Zero pedidos duplicados no cenário testado.
RNF-07	Segurança	Sem exposição desnecessária de dados pessoais.
RNF-08	LGPD	Finalidade clara e consentimento quando aplicável.
RNF-09	Observabilidade	Falhas simuladas registradas em log ou auditoria.
RNF-10	Manutenibilidade	Regras críticas testáveis isoladamente.
RNF-11	Escalabilidade planejada	Carga de referência com até 20 usuários.
RNF-12	Acessibilidade básica	Controles e mensagens compreensíveis.

2.5 Requisitos de qualidade mensuráveis

ID	Dimensão	Meta	Evidência esperada
RQ-01	Desempenho do cardápio	P95 <= 2s	Relatório de API ou carga.
RQ-02	Desempenho do pedido	P95 <= 2s	Relatório de API ou carga.
RQ-03	Confiabilidade	0 duplicidades no cenário testado	Teste de idempotência.
RQ-04	Funcionalidade	0 inconsistências no cardápio testado	Teste de produto indisponível.
RQ-05	Usabilidade	Até 3 etapas principais após seleção de produtos	Checklist e print da jornada.
RQ-06	Responsividade	Sem quebra crítica nos viewports alvo	Prints mobile ou E2E.

ID	Dimensão	Meta	Evidência esperada
RQ-07	Segurança e LGPD	Checklist obrigatório atendido	Aviso, consentimento e logs.
RQ-08	Observabilidade	100% das falhas simuladas rastreadas	Evento de auditoria e mensagem.

2.6 Critérios de aceitação

ID	Dado que	Quando	Então
CA-01	Cliente no início do fluxo	Seleciona unidade ativa	O cardápio correspondente fica disponível.
CA-02	Unidade selecionada	Cardápio é carregado	Apenas produtos disponíveis aparecem.
CA-03	Produto fora do cardápio da unidade	Cliente tenta usá-lo no pedido	Inclusão é rejeitada com mensagem.
CA-04	Carrinho vazio	Cliente tenta confirmar	Pedido não é criado.
CA-05	Item com quantidade ≤ 0	Pedido é validado	Quantidade é rejeitada.
CA-06	Pedido válido	Cliente revisa antes de pagar	Itens, quantidades e total são exibidos.
CA-07	Pedido válido	Pagamento retorna APROVADO	Pedido fica confirmado.
CA-08	Pedido válido	Pagamento retorna NEGADO	Pedido não fica confirmado e mensagem é exibida.
CA-09	Serviço de pagamento falha	Pedido é enviado	Erro controlado e rastreio são gerados.
CA-10	Pedido existente	Cliente consulta status	Status atual é retornado.
CA-11	Cliente repete confirmação	Mesmo idempotencyKey é usado	Pedido não é duplicado.
CA-12	Dado pessoal é solicitado	Interface exibe finalidade	Consentimento é claro e registrável.

3. Modelagem e Arquitetura

3.1 Decisão arquitetural

O projeto adota um monólito modular orientado por domínios, inspirado em DDD leve. A decisão busca alta coesão, baixo acoplamento, clareza de manutenção, rastreabilidade e separação de frentes de atuação. A arquitetura não propõe microsserviços nem DDD completo, pois o recorte acadêmico exige implementação parcial e foco em QA.

Decisão: organizar backend por módulos de domínio e frontend por pages, sections, features e shared. A API REST atua como fronteira entre as aplicações.

3.2 Visão macro da solução

```

[Cliente mobile]
  |
  v
[Frontend React + TypeScript]
  |
  v
[API REST Flask]
  |
  +--> [Unidades]
  +--> [Cardápio]
  +--> [Pedidos] ---> [Pagamentos]
```

```

+--> [Auditoria]
+--> [Privacidade]
|
v
[SQLite + seed controlado]
|
v
[Testes, logs, métricas e evidências]

```

3.3 Contextos de domínio

Domínio	Responsabilidade	Fronteira
Unidade	Unidades ativas, tipo de operação e contexto da retirada.	Não calcula pedido nem pagamento.
Cardápio	Produtos e disponibilidade por unidade.	Não confirma pedido.
Pedido	Itens, total, status, idempotência e orquestração.	Não processa pagamento real.
Pagamento	Simulação de aprovado, negado e falha.	Não conhece regra de cardápio.
Auditoria	Eventos técnicos e operacionais rastreáveis.	Não altera regra de negócio.
Privacidade	Avisos, consentimentos e minimização de dados.	Não exige cadastro completo para pedido simples.

3.4 Estrutura do backend

```

backend/
  app/
    __init__.py
  system/
    controller.py
  config/
    settings.py
    database.py
  common/
    errors.py
    enums.py
  shared/
    utils/
      ids.py
      time.py
  modules/
    unidade/
      controller.py
      dto.py
      repository.py
      service.py
    cardapio/
      controller.py
      dto.py
      repository.py
      service.py
    pedido/

```



```
    controller.py
    dto.py
    repository.py
    service.py
pagamento/
    controller.py
    dto.py
    repository.py
    service.py
auditoria/
    dto.py
    repository.py
    service.py
privacidade/
    dto.py
    repository.py
    service.py
tests/
    unit/
    integration/
```

3.5 Estrutura do frontend

```
frontend/src/
  app/
    App.tsx
  shared/
    components/
    styles/
    types/
  services/
    apiClient.ts
  features/
    unidades/
    cardapio/
    pedido/
    pagamento/
    privacidade/
  pages/
    pedido-retirada/
      PedidoRetiradaPage.tsx
      usePedidoRetiradaFlow.ts
      components/
      sections/
        SelecionarUnidadeSection.tsx
        CardapioSection.tsx
        CarrinhoSection.tsx
        ConfirmacaoPedidoSection.tsx
        StatusPedidoSection.tsx
        PrivacidadeSection.tsx
```

3.6 Contrato mínimo da API

Método	Endpoint	Responsabilidade
GET	/api/health	Verificar disponibilidade local da API.
GET	/api/unidades	Listar unidades ativas.
GET	/api/unidades/{id}/cardapio	Exibir produtos disponíveis na unidade.
POST	/api/pedidos	Criar pedido e executar pagamento simulado.
GET	/api/pedidos/{id}/status	Consultar status atual.
POST	/api/pagamentos/simular	Exercitar simulação direta de pagamento em testes.

3.7 Modelo de dados mínimo

Entidade	Campos essenciais	Finalidade
Unidade	id, nome, tipo, ativa	Identificar local de retirada.
Produto	id, nome, preço, ativo, sazonal	Representar item de cardápio.
CardapioUnidade	unidadeId, produtoId, disponível	Controlar disponibilidade local.
Pedido	id, unidadeId, status, valorTotal, idempotencyKey	Guardar o fluxo de compra.
ItemPedido	pedidoId, produtoId, quantidade, subtotal	Guardar itens válidos.
PagamentoSimulado	pedidoId, resultado, mensagem	Registrar retorno controlado.
AuditoriaEvento	tipo, entidadeId, nível, mensagem	Evidenciar eventos relevantes.
Consentimento	pedidoId, finalidade, aceite	Registrar escolha de privacidade.

3.8 Estratégia de testes e ferramentas

A estratégia de testes foi definida para validar regras de negócio, integração entre módulos, experiência do usuário, segurança/privacidade e desempenho. Como a trilha é Qualidade de Software, o foco é garantir que cada teste esteja vinculado a um risco, requisito, critério de aceitação e evidência.

Tipo de teste	Aplicação no projeto	Ferramentas previstas
Unitário	Regras isoladas de pedido, pagamento, idempotência e carrinho.	pytest, Vitest
Integração	API, persistência SQLite, pagamentos simulados e auditoria.	Flask test client, pytest
Sistema/E2E	Jornada de unidade, cardápio, carrinho, pagamento e status.	Playwright ou Selenium
Regressão	Reexecução dos fluxos críticos após alterações.	pytest, Vitest, Playwright
Desempenho	Carga em cardápio e criação de pedido.	k6, JMeter
Segurança/LGPD	Validação de entrada, exposição de dados e consentimento.	Checklist, OWASP ZAP como possibilidade
Responsividade	Validação em 360px	DevTools, Playwright screenshots
UAT/Usabilidade	Clareza da jornada, mensagens e conclusão de tarefa.	Checklist de homologação

4. Entrega Técnica

4.1 Recorte técnico entregue

Área	Entrega prevista/realizada	Evidência a inserir
Backend	API Flask, organização modular, SQLite, seed, regras de pedido e pagamento simulado.	Print da estrutura de pastas, terminal rodando e relatório pytest.
Frontend	React + TypeScript, fluxo mobile-first de pedido para retirada, sections e features.	Print da tela inicial, cardápio, carrinho, status e build/testes.
Testes	Testes unitários, integração, E2E inicial e carga planejada.	Relatórios, terminal, prints e logs.
Documentação	Documentos canônicos de visão, requisitos, arquitetura, API, QA e execução.	Print ou listagem do diretório docs.

4.2 Estrutura sugerida do repositório final

```
raizes-nordeste-qa/
docs/
backend/
frontend/
README.md
```

Antes de empacotar a entrega final, recomenda-se remover pastas de dependências e temporários como .env, node_modules, dist e caches, mantendo apenas os arquivos necessários para reprodução.

4.3 Como executar localmente

Backend:

```
cd backend
python -m venv .env
.\.env\Scripts\python.exe -m pip install -r requirements.txt
.\.env\Scripts\python.exe -m pytest --cov=app --cov-report=term-missing
.\.env\Scripts\python.exe -m flask --app app run --host 127.0.0.1 --port 5000
```

Frontend:

```
cd frontend
npm install
npm test
npm run build
npm run test:e2e
npm run dev -- --host 127.0.0.1 --port 5173
```

4.4 Quadros de evidência da entrega técnica

EV-001 - Estrutura do repositório e arquitetura modular

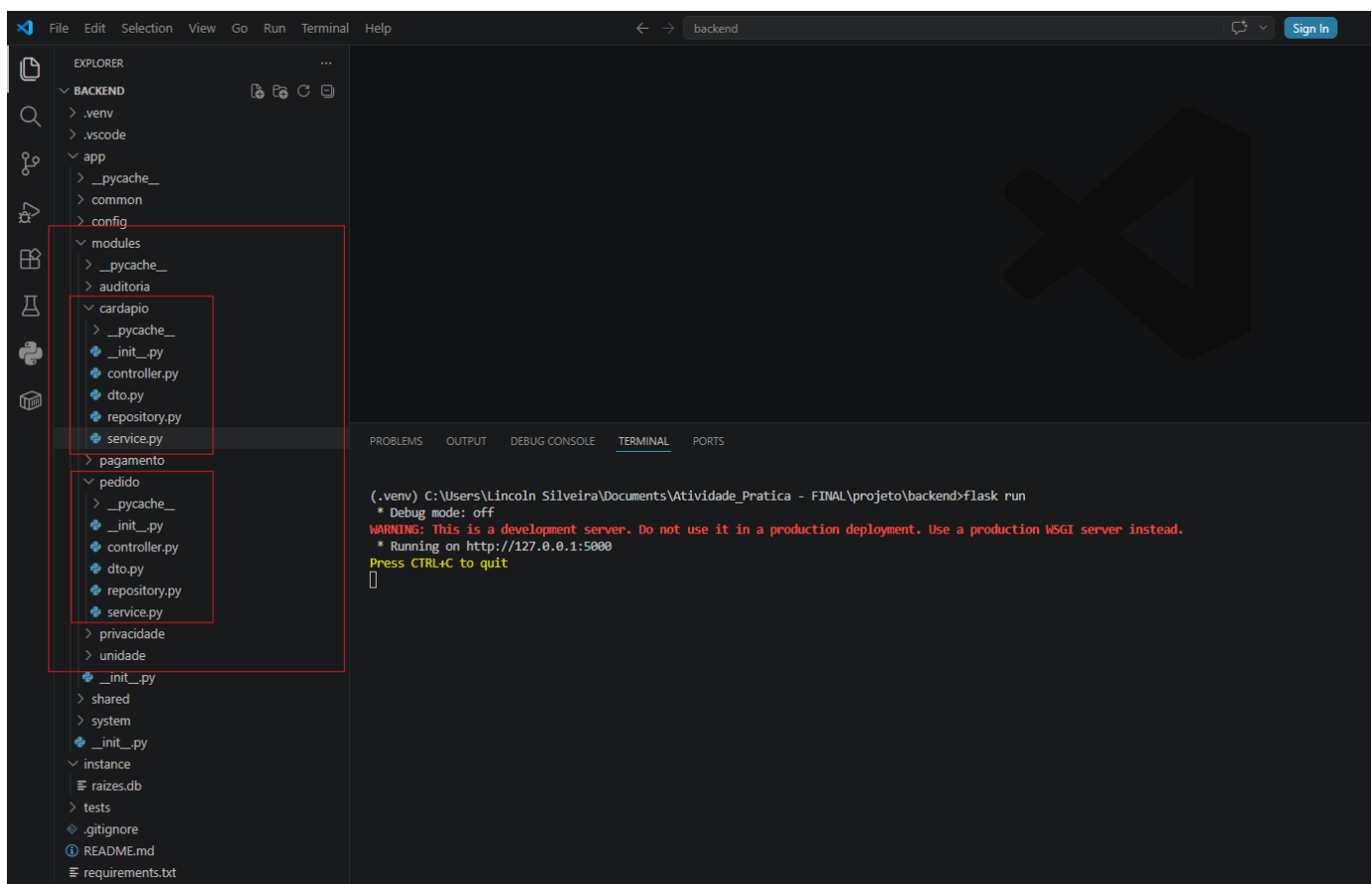
Cenário relacionado	Organização estrutural da solução
Requisito relacionado	RNF-10 — Manutenibilidade / RQ-08 — Observabilidade e rastreabilidade

Resultado esperado	Projeto organizado em backend, frontend, documentação e evidências, com separação modular por domínio e por responsabilidade
Resultado obtido	Estrutura organizada com backend modular por domínios de negócio, frontend separado por página/seções/fluxo e documentação canônica do projeto
Status	Aprovado

Espaço para evidência/print:

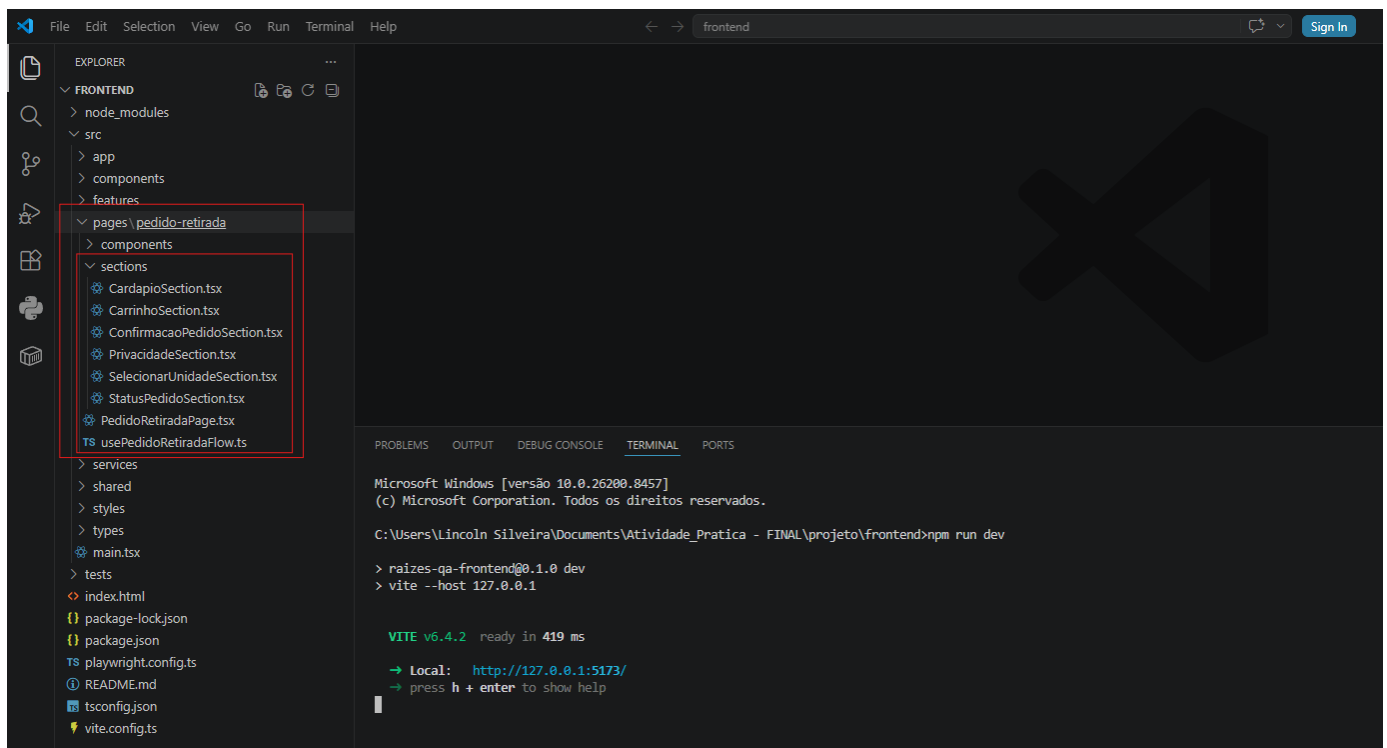
Backend - Estrutura modular do backend:

Evidência da organização do backend por módulos de domínio, separando responsabilidades como cardápio, pedido, pagamento, privacidade, unidade e auditoria.



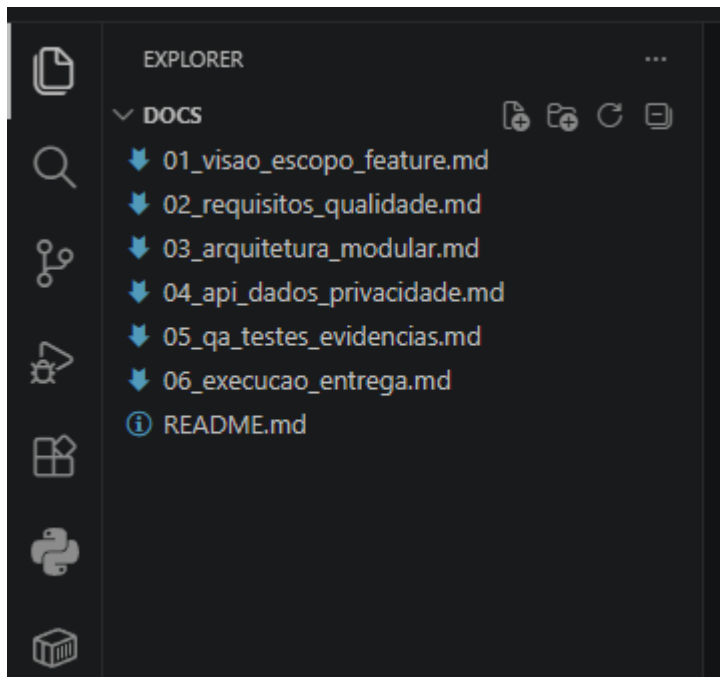
Frontend - Estrutura modular:

Evidência da organização da página de pedido para retirada em seções específicas, favorecendo manutenção, legibilidade e evolução da interface.



Documentações canônicas:

Evidência dos documentos principais usados para orientar visão, requisitos, arquitetura, API, QA, testes, evidências e execução da entrega.



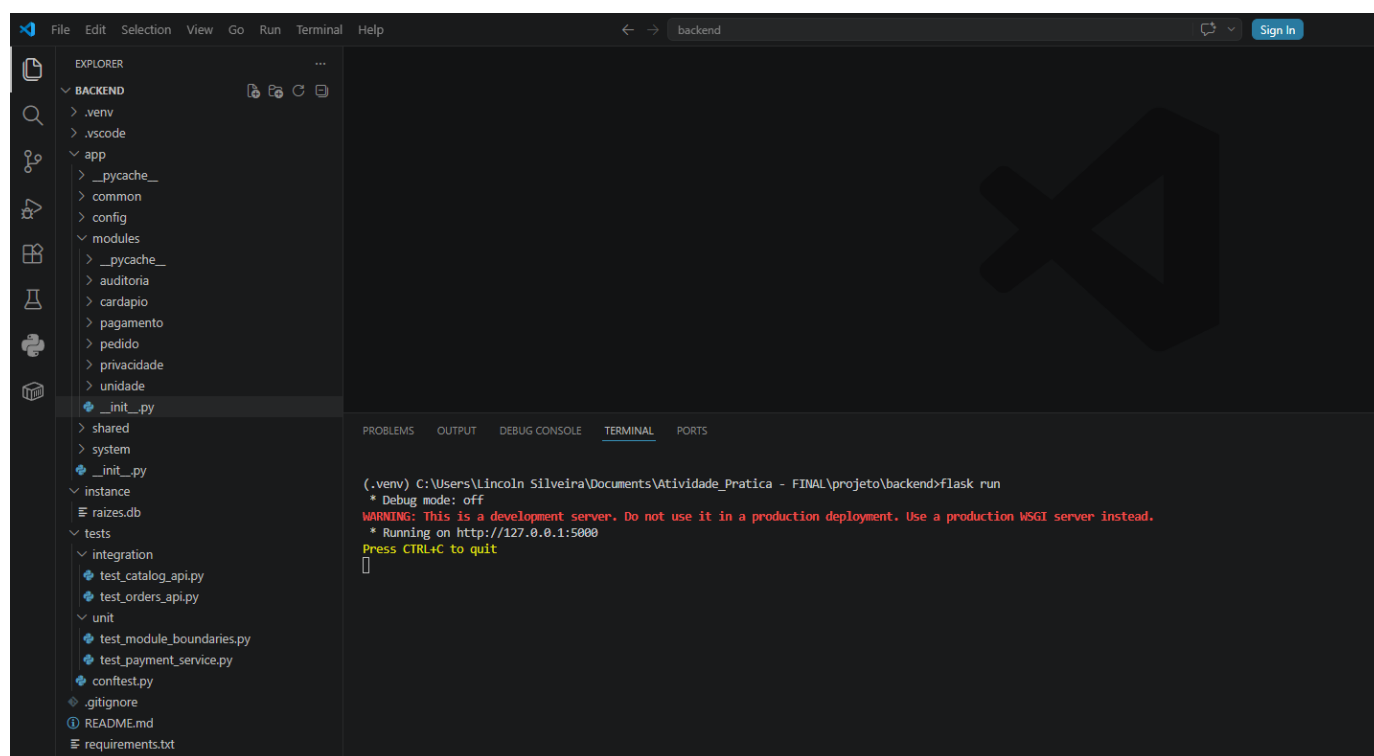
Estrutura do repositório e arquitetura modular.

Evidência da organização do projeto em backend, frontend, documentação e evidências, com backend estruturado por módulos de domínio para favorecer manutenção, testabilidade e evolução.

EV-002 - Backend em execução

Cenário relacionado	Execução local da API
Requisito relacionado	RNF-10 — Manutenibilidade / RNF-09 — Observabilidade / Entrega técnica backend
Resultado esperado	Backend iniciado corretamente e disponível em ambiente local
Resultado obtido	API Flask executada com sucesso em http://127.0.0.1:5000
Status	Aprovado

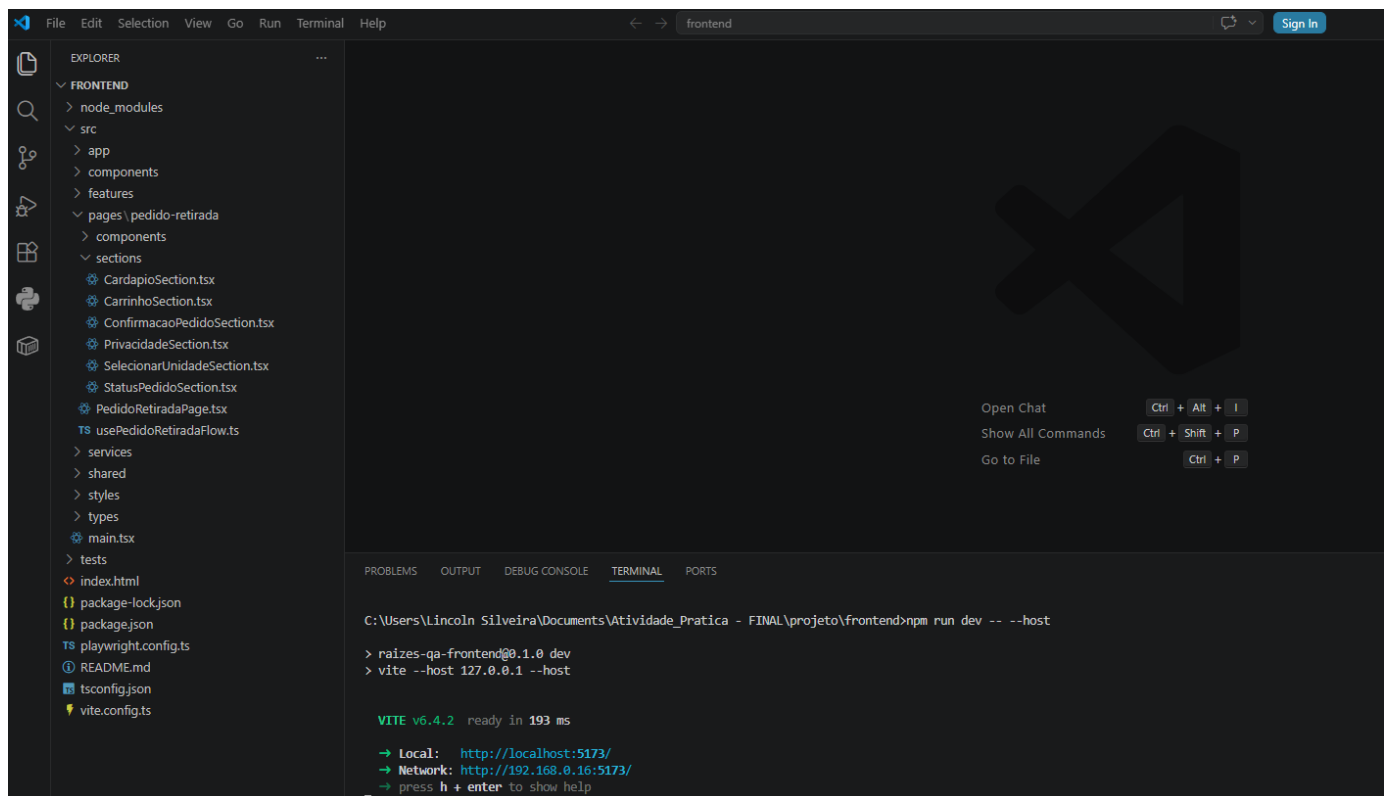
Execução com ambiente virtual Python ativo, banco SQLite disponível e estrutura modular do projeto visível no VS Code.



EV-003 - Frontend em execução

Cenário relacionado	Execução local da interface web
Requisito relacionado	RNF-04 — Usabilidade / RNF-05 — Responsividade / RQ-05 — Usabilidade
Resultado esperado	Frontend iniciado corretamente e acessível no navegador
Resultado obtido	Aplicação React/Vite executada com sucesso em http://127.0.0.1:5173/
Status	Aprovado

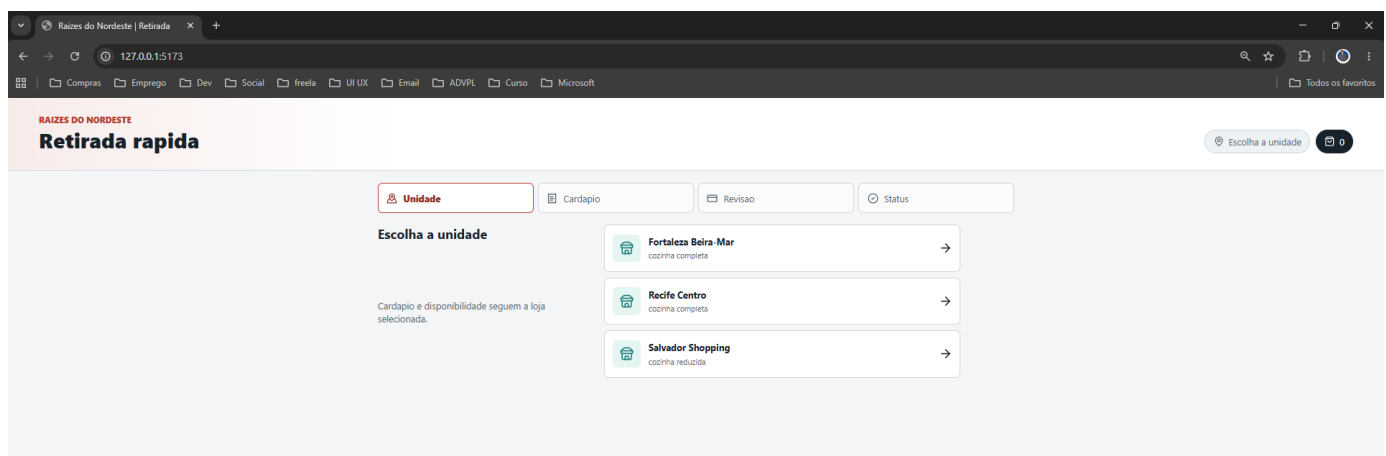
Execução local do frontend da solução. A interface foi desenvolvida em React com TypeScript e disponibilizada localmente pelo Vite, permitindo a execução da jornada de pedido para retirada rápida com integração ao backend Flask.



Interface Web:

Aplicação web aberta no navegador, permitindo o início da jornada de pedido para retirada rápida.

Desktop:



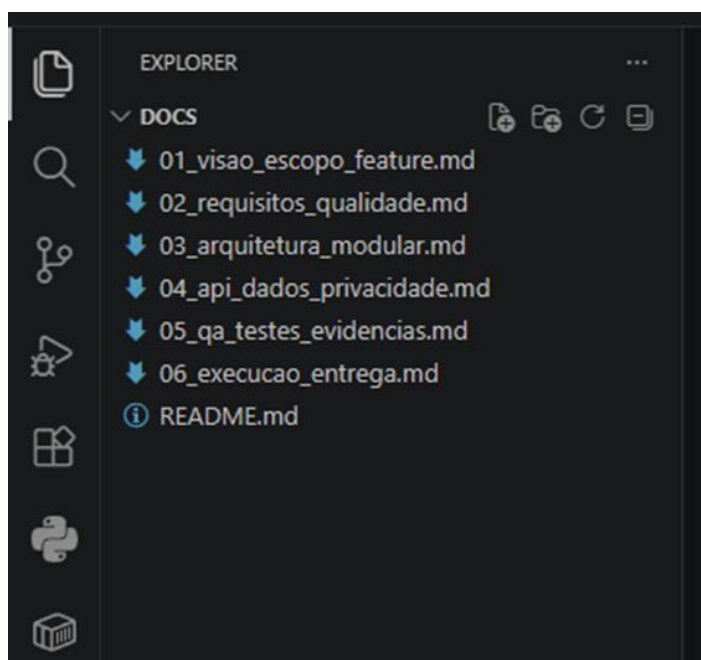
EV-004 - Documentação do projeto

Cenário relacionado	Organização documental da entrega
---------------------	-----------------------------------

Requisito relacionado	Documentação formal / RQ-08 — Observabilidade e rastreabilidade / RNF-10 — Manutenibilidade
Resultado esperado	Projeto documentado com visão, requisitos, arquitetura, API, QA, testes, evidências e execução
Resultado obtido	Documentação canônica organizada em arquivos Markdown, cobrindo as principais decisões e artefatos da entrega
Status	Aprovado

Espaço para evidência/print:

Organização da documentação canônica do projeto. Os documentos foram estruturados para registrar a visão do escopo, requisitos de qualidade, arquitetura modular, API, dados, privacidade, estratégia de QA, testes, evidências e orientações de execução da entrega.



5. Plano de Testes e Evidências

5.1 Diretriz central de QA

risco -> requisito -> critério de aceitação -> teste -> evidência -> métrica

A validação será considerada adequada quando os cenários críticos possuírem entrada, passos, saída esperada, resultado obtido e evidência anexada. Os testes positivos comprovam o fluxo esperado; os negativos comprovam o tratamento de erro, a resiliência e a qualidade da experiência.

5.2 Plano de Garantia da Qualidade

O Plano de Garantia da Qualidade define como a qualidade será planejada, verificada, validada e evidenciada no projeto. Para esta entrega, o QA Plan foi aplicado sobre a feature de pedido para retirada rápida com pagamento simulado, conectando requisitos, critérios de aceitação, cenários de teste, métricas e evidências.

5.2.1 Plano de Qualidade

Item	Diretriz adotada
Objetivo de qualidade	Garantir que a feature de pedido para retirada rápida seja funcional, confiável, rastreável, responsiva, segura quanto aos dados pessoais e validada por evidências técnicas.
Escopo da qualidade	Requisitos funcionais, requisitos não funcionais, critérios de aceitação, testes automatizados, testes manuais, evidências, métricas, LGPD e rastreabilidade.
Critério de aprovação	A entrega será considerada aprovada quando os cenários críticos possuírem resultado esperado, resultado obtido, evidência registrada e status definido.
Critério de reprovação	A entrega será considerada reprovada ou pendente quando houver falha crítica sem tratamento, ausência de evidência ou divergência entre requisito, teste e resultado obtido.
Controle de qualidade	Uso de matriz de rastreabilidade, checklist de evidências, testes automatizados, prints, logs, relatórios e validação manual da jornada.
Qualidade esperada	Sistema compreensível, testável, modular, com baixa exposição de dados pessoais, mensagens claras, desempenho dentro da meta e comportamento consistente nos fluxos críticos.

5.2.2 Plano de Testes

Tipo de teste	Objetivo	Ferramenta/evidência
Testes unitários	Validar regras isoladas de domínio, como pagamento simulado, idempotência e validações de pedido.	pytest, Vitest, prints de terminal e relatórios.
Testes de integração	Validar comunicação entre API, módulos, persistência SQLite e regras de negócio.	pytest, Flask test client e evidências de API.
Testes de sistema/E2E	Validar a jornada completa do usuário, da seleção da unidade até o status final do pedido.	Playwright e prints do fluxo.
Testes de regressão	Reexecutar testes após correções ou ajustes para garantir que funcionalidades já validadas não foram afetadas.	pytest, Vitest e Playwright.
Testes negativos	Validar comportamento em pedido vazio, quantidade inválida, pagamento negado, falha de pagamento e produto indisponível.	Postman/Insomnia, interface, logs e testes automatizados.
Testes de desempenho	Medir tempo de resposta e taxa de falha dos endpoints principais sob carga controlada.	k6 e relatório de execução.
Testes de responsividade	Verificar a experiência em viewport mobile, garantindo legibilidade e ausência de quebra crítica.	DevTools e prints mobile.
Testes de LGPD/privacidade	Validar aviso de finalidade, consentimento e ausência de coleta de dados sensíveis desnecessários.	Prints da interface e checklist LGPD.
Testes de aceitação/usabilidade	Avaliar clareza da jornada, compreensão da unidade selecionada, cardápio, carrinho, status e mensagens.	Checklist UAT/usabilidade.

5.2.3 Cronograma de Validação

Etapas	Atividade	Entregável	Status
1	Revisão do estudo de caso e definição do recorte da feature.	Feature de pedido para retirada rápida definida.	Concluído
2	Levantamento de requisitos, riscos e critérios de aceitação.	Requisitos funcionais, não funcionais e critérios documentados.	Concluído
3	Definição da arquitetura modular e contratos mínimos da API.	Estrutura backend/frontend e contrato da API documentados.	Concluído
4	Implementação parcial da fatia vertical testável.	Backend, frontend, banco SQLite e fluxo principal.	Concluído
5	Execução dos testes unitários e de integração do backend.	Relatório pytest e cobertura de testes.	Concluído
6	Execução dos testes frontend, build e E2E.	Relatórios Vitest, build Vite e Playwright.	Concluído

7	Execução dos testes manuais e negativos.	Prints de pagamento negado, falha, pedido inválido e produto indisponível.	Concluído
8	Validação de responsividade, LGPD e usabilidade.	Prints mobile, evidência LGPD e checklist UAT.	Concluído
9	Execução de teste de desempenho/carga.	Relatório k6 com tempo de resposta e taxa de falha.	Concluído
10	Consolidação das evidências e revisão final do documento.	Documento final com matriz, métricas e evidências.	Concluído

5.2.4 Papéis e Responsabilidades

Papel	Responsabilidade no projeto
Analista de requisitos/QA	Interpretar o estudo de caso, definir riscos, requisitos, critérios de aceitação e matriz de rastreabilidade.
Desenvolvedor backend	Implementar API, módulos de domínio, regras de pedido, pagamento simulado, persistência e testes backend.
Desenvolvedor frontend	Implementar a jornada de pedido, organização por páginas/seções/features, responsividade e integração com a API.
Responsável por testes automatizados	Criar e executar testes unitários, integração, frontend, build e E2E.
Responsável por testes manuais/UAT	Validar a jornada do usuário, clareza da interface, mensagens, status, responsividade e usabilidade.
Responsável por privacidade/LGPD	Verificar finalidade de uso dos dados, consentimento, minimização e ausência de dados sensíveis desnecessários.
Responsável por evidências	Coletar prints, logs, relatórios de execução, resultados de testes e consolidar no documento final.
Avaliador/professor	Verificar aderência ao roteiro, clareza da solução, coerência técnica e qualidade das evidências apresentadas.

5.3 Matriz de rastreabilidade

A matriz de rastreabilidade consolida a relação entre requisitos, riscos, cenários de teste, evidências coletadas e resultado obtido. Essa estrutura permite demonstrar que a validação não foi feita de forma isolada, mas conectando necessidade de negócio, critério de qualidade, teste executado e evidência registrada.

Requisito	Risco	Teste	Evidência esperada	Status
RF-01 — Permitir seleção de unidade antes do pedido	Cliente iniciar pedido sem contexto da loja	CT-01 — Pedido aprovado	EV-005 / EV-013 — Unidade selecionada no fluxo principal e validada no E2E	Aprovado
RF-02 — Exibir cardápio da unidade selecionada	Exibição de produtos incorretos para a unidade	CT-01 / CT-04	EV-005 / EV-008 — Cardápio exibido conforme a unidade selecionada e regra de disponibilidade validada	Aprovado
RF-03 — Permitir adicionar produtos disponíveis ao pedido	Cliente não conseguir montar um pedido válido	CT-01 — Pedido aprovado	EV-005 / EV-013 — Produtos disponíveis adicionados ao carrinho e fluxo concluído com sucesso	Aprovado
RF-05 — Rejeitar produto indisponível para a unidade	Pedido aceito com produto não disponível na loja	CT-04 — Produto indisponível	EV-008 — Pedido com produto indisponível rejeitado com mensagem de erro	Aprovado
RF-06 — Impedir pedido vazio	Pedido inválido criado sem itens	CT-05 — Pedido vazio	EV-009 — Pedido sem itens recusado por validação	Aprovado

Requisito	Risco	Teste	Evidência esperada	Status
RN-04 / CA-05 — Rejeitar quantidade inválida	Pedido com quantidade zero ou negativa	CT-06 — Quantidade inválida	EV-009 — Item com quantidade inválida rejeitado com erro controlado	Aprovado
RF-07 — Exibir revisão com itens, quantidades e total	Cliente confirmar pedido sem compreender o resumo	CT-01 — Pedido aprovado	EV-005 / EV-014 — Tela de revisão exibiu itens, quantidades e valor total	Aprovado
RF-08 — Calcular valor total a partir dos itens válidos	Valor exibido incorreto ou inconsistente	CT-01 — Pedido aprovado	EV-005 — Total do pedido exibido corretamente no resumo	Aprovado
RF-09 — Solicitar pagamento simulado após confirmação	Pedido confirmado sem passar pela etapa de pagamento	CT-01 / CT-02 / CT-03	EV-005 / EV-006 / EV-007 — Fluxo apresentou cenários de pagamento aprovado, negado e falha	Aprovado
RF-10 — Registrar retorno APROVADO, NEGADO ou FALHA	Status não refletir o retorno do pagamento	CT-01 / CT-02 / CT-03	EV-005 / EV-006 / EV-007 — Sistema tratou os três retornos simulados de pagamento	Aprovado
RF-11 — Confirmar pedido apenas com pagamento aprovado	Pedido confirmado indevidamente após negativa ou falha	CT-01 / CT-02 / CT-03	EV-005 / EV-006 / EV-007 — Apenas pagamento aprovado concluiu o pedido; negativa e falha não confirmaram indevidamente	Aprovado
RF-12 — Exibir mensagem clara para pagamento negado	Usuário sem orientação após negativa	CT-02 — Pagamento negado	EV-006 — Sistema exibiu mensagem/status coerente para pagamento negado	Aprovado
RF-13 — Exibir mensagem clara e rastreável para falha de pagamento	Falha técnica sem tratamento compreensível	CT-03 — Falha de pagamento	EV-007 — Falha de pagamento tratada com mensagem e rastreio/log	Aprovado
RF-14 — Permitir consulta de status do pedido	Cliente não conseguir acompanhar o resultado do pedido	CT-07 — Consulta de status	EV-005 / EV-013 — Status final do pedido exibido ao usuário	Aprovado
RF-16 / RQ-03 — Evitar duplicidade na confirmação	Pedido duplicado por repetição de ação	CT-08 — Clique duplicado / idempotência	EV-010 / EV-011 — Testes backend validaram comportamento de idempotência	Aprovado
RF-17 — Exibir aviso de privacidade	Uso de dados pessoais sem clareza	CT-12 — Consentimento aplicável	EV-015 — Interface exibiu aviso de privacidade com finalidade de nome e telefone	Aprovado
RF-18 — Registrar consentimento quando aplicável	Consentimento ausente ou ambíguo	CT-12 — Consentimento aplicável	EV-015 — Checkbox de autorização para comunicação sobre o pedido exibido no fluxo	Aprovado
RNF-01 / RQ-01 — Desempenho do cardápio	Lentidão no carregamento do cardápio em horário de pico	CT-09 — Cardápio dentro da meta	EV-016 — k6 obteve P95 de 7,91ms e 0% de falhas HTTP	Aprovado
RNF-02 / RQ-02 — Desempenho da criação de pedido/API	Atraso nas operações principais do fluxo	CT-09 / CT-10	EV-016 — Carga controlada validou endpoints principais com P95 abaixo de 2 segundos	Aprovado
RNF-04 / RQ-05 — Usabilidade	Jornada confusa ou difícil de concluir	CT-01 / UAT	EV-005 / EV-017 — Fluxo principal validado e checklist UAT/usabilidade aprovado	Aprovado
RNF-05 / RQ-06 — Responsividade mobile	Interface quebrada em dispositivos móveis	CT-11 — Fluxo mobile	EV-014 — Prints mobile demonstraram jornada sem quebra crítica	Aprovado

Requisito	Risco	Teste	Evidência esperada	Status
RNF-07 / RNF-08 / RQ-07 — Segurança e LGPD	Coleta excessiva ou falta de consentimento	CT-12 — LGPD e privacidade	EV-015 — Aviso de finalidade, consentimento e ausência de dados financeiros reais evidenciados	Aprovado
RNF-09 / RQ-08 — Observabilidade	Falha sem rastreo técnico ou operacional	CT-03 — Falha de pagamento	EV-007 — Falha de pagamento gerou mensagem e rastreo/log para análise	Aprovado
RNF-10 — Manutenibilidade	Dificuldade de evolução, manutenção e teste	Arquitetura modular / testes automatizados	EV-001 / EV-010 / EV-011 / EV-012 — Backend e frontend modularizados; testes executados com sucesso	Aprovado
Cobertura de testes	Regras críticas sem validação automatizada	Testes backend	EV-010 / EV-011 — 15 testes backend aprovados e 95% de cobertura total	Aprovado
Build e testes frontend	Interface com erro de compilação ou regressão visual básica	Testes frontend/build	EV-012 — 3 testes frontend aprovados e build concluído com sucesso	Aprovado
Sistema ponta a ponta	Integração frontend/backend não validada	CT-01 — Teste E2E	EV-013 — Teste Playwright executado com 1 teste aprovado	Aprovado

5.4 Métricas e indicadores

Métrica	Meta	Coleta	Resultado obtido
Taxa de defeitos críticos	<= 2% em referência simulada	k6/testes automatizados	0% de falhas HTTP no teste de carga
Cobertura de testes	>= 80% nas regras críticas	Relatório pytest-cov	Backend com 95% de cobertura total e Frontend com 3 testes automatizados aprovados e build concluído sem erros.
Tempo de resposta	P95 <= 2s em cardápio e pedido	k6	P95 de 7,91ms nos endpoints de unidades e cardápio
Disponibilidade	99,5% como referência	Meta de qualidade para produção	A disponibilidade produtiva não foi mensurada por não haver ambiente em produção. Em ambiente local, backend e frontend permaneceram acessíveis durante a execução das evidências e testes.
Satisfação do usuário	>= 85% em UAT/usabilidade	Checklist UAT/usabilidade	Não houve pesquisa formal com usuários reais. Como validação de aceitação, o checklist UAT/usabilidade foi aprovado nos critérios avaliados.
MTTR	<= 24h para defeito crítico como meta teórica	Meta de processo para produção	Não mensurado como indicador operacional. O projeto não possui ciclo produtivo de incidentes; a métrica foi mantida como referência para

Métrica	Meta	Coleta	Resultado obtido
			evolução futura do processo de QA.

5.5 Evidências por tipo de teste

5.5.1 Cenários de teste

ID	Cenário	Tipo	Entradas	Passos resumidos	Saída esperada
CT-01	Pedido aprovado	Positivo	Unidade ativa, produto disponível, pagamento APROVADO.	Selecionar unidade, escolher item, revisar, confirmar.	Pedido CONFIRMADO e status visível.
CT-02	Pagamento negado	Negativo	Pedido válido com pagamento NEGADO.	Confirmar pedido com cenário NEGADO.	Pedido não confirmado e mensagem clara.
CT-03	Falha de pagamento	Negativo	Pedido válido com cenário FALHA.	Confirmar pedido simulando falha.	Erro controlado, rastreo e auditoria.
CT-04	Produto indisponível	Negativo	Produto não pertence à unidade.	Tentar criar pedido com item indisponível.	Item rejeitado e erro exibido.
CT-05	Pedido vazio	Negativo	Carrinho sem itens.	Tentar confirmar pedido.	Pedido bloqueado/erro 400.
CT-06	Quantidade inválida	Negativo	Item com quantidade <= 0.	Enviar pedido inválido.	Validação rejeita item.
CT-07	Consulta de status	Positivo	Pedido já criado.	Consultar /api/pedidos/{id}/status.	Status atual retornado.
CT-08	Clique duplicado	Regressão	Mesmo idempotencyKey.	Enviar duas confirmações.	Pedido não duplicado.
CT-09	Cardápio dentro da meta	Desempenho	Endpoint de cardápio.	Executar carga.	P95 <= 2s.
CT-10	Pedido dentro da meta	Desempenho	Endpoint de unidades e cardápio.	Executar carga.	P95 <= 2s.
CT-11	Fluxo mobile	Responsividade	Viewport 360px	Executar jornada visual.	Sem quebra crítica.
CT-12	Consentimento aplicável	LGPD	Dado pessoal ou comunicação.	Exibir aviso e registrar escolha.	Finalidade clara e registro coerente.

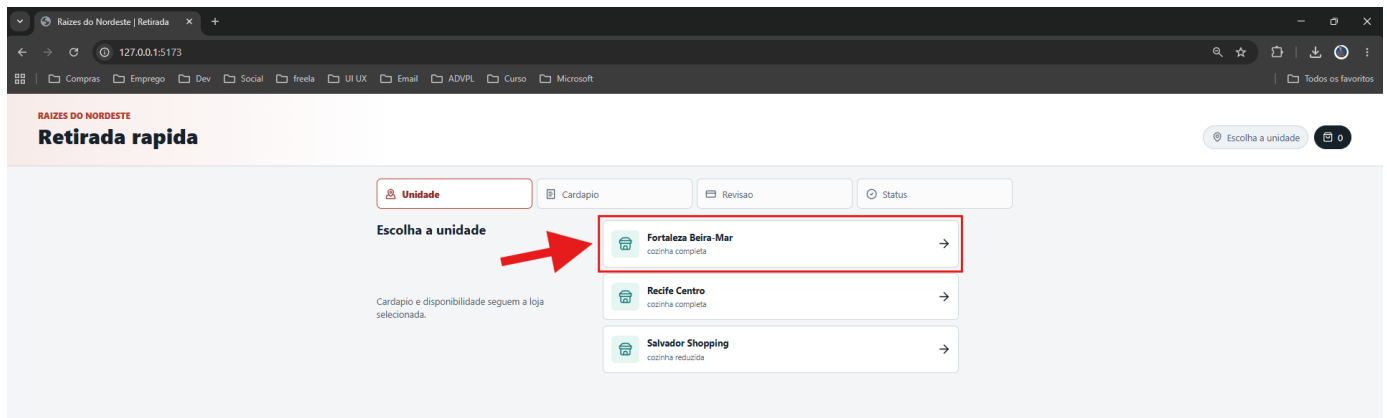
EV-005 - Fluxo aprovado ponta a ponta

Cenário relacionado	CT-01 — Pedido aprovado
Requisito relacionado	RF-01, RF-02, RF-03, RF-07, RF-08, RF-09, RF-10, RF-11, RF-14 / RQ-05
Resultado esperado	O cliente deve conseguir selecionar uma unidade, escolher produtos disponíveis, revisar o pedido, confirmar pagamento aprovado e visualizar o status final
Resultado obtido	Pedido concluído com pagamento simulado aprovado e status final exibido ao usuário
Status	Aprovado

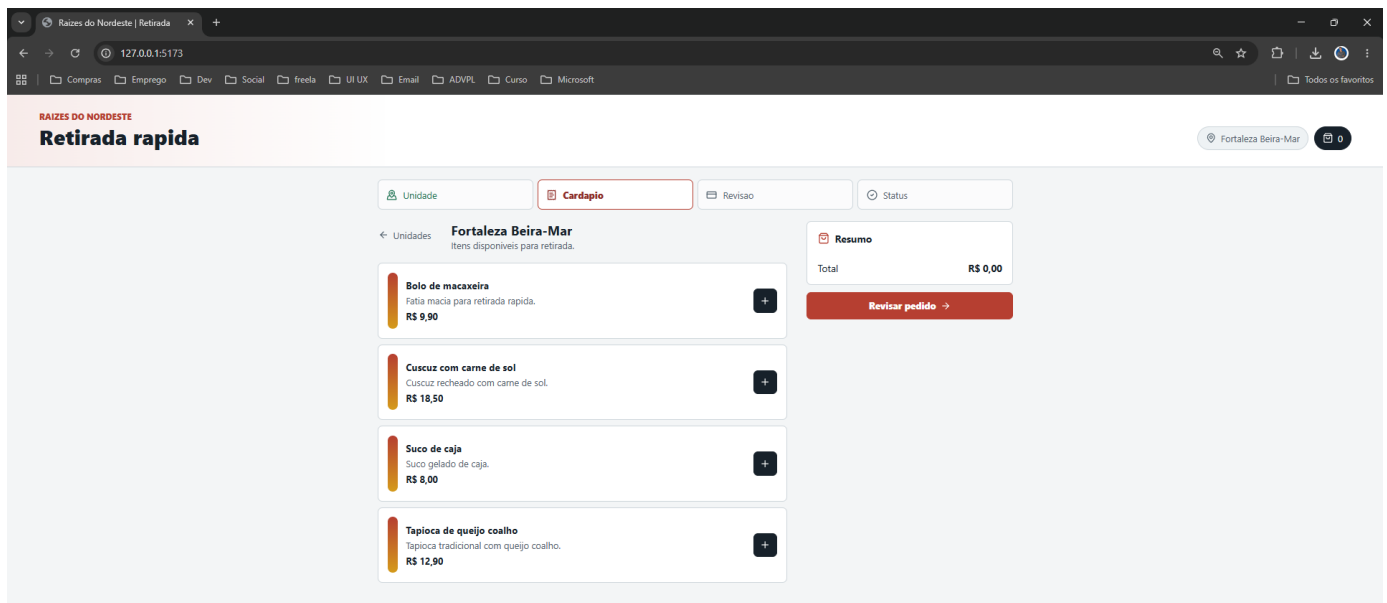
A evidência a seguir apresenta o fluxo aprovado ponta a ponta da feature de pedido para retirada rápida. O cenário válida a seleção de unidade, exibição do cardápio correspondente, inclusão de produtos disponíveis, revisão do pedido, simulação de pagamento aprovado e exibição do status final ao usuário.

Desktop:

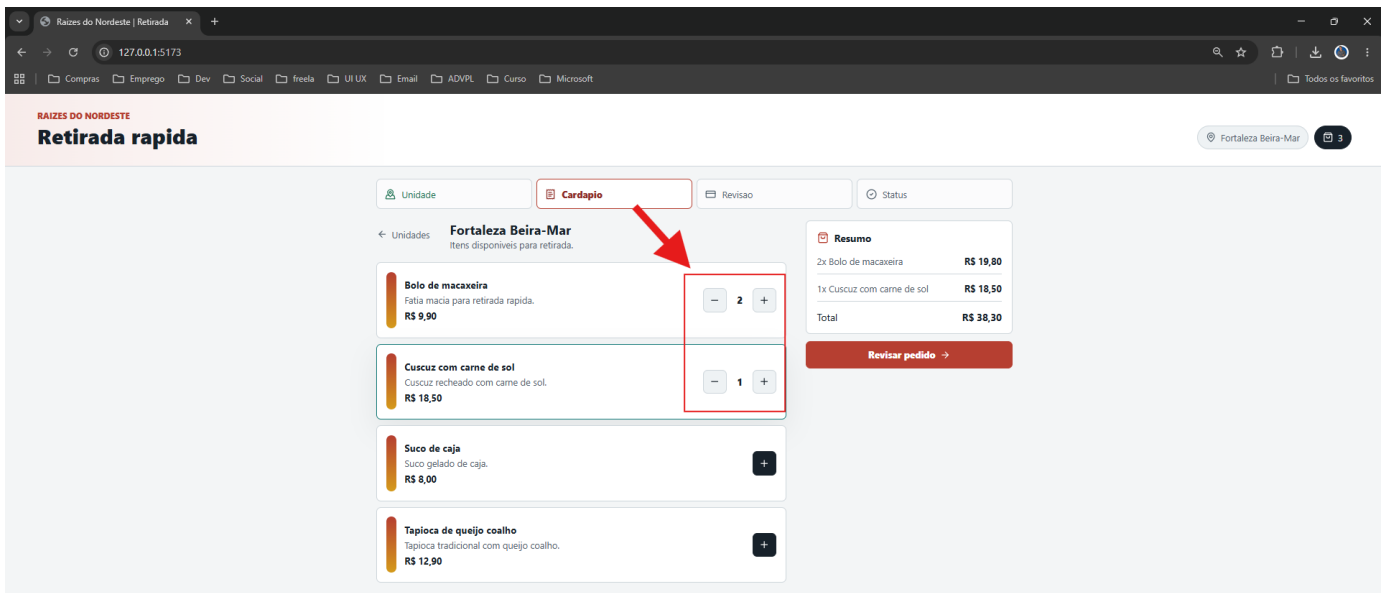
1 - Seleção de unidade:



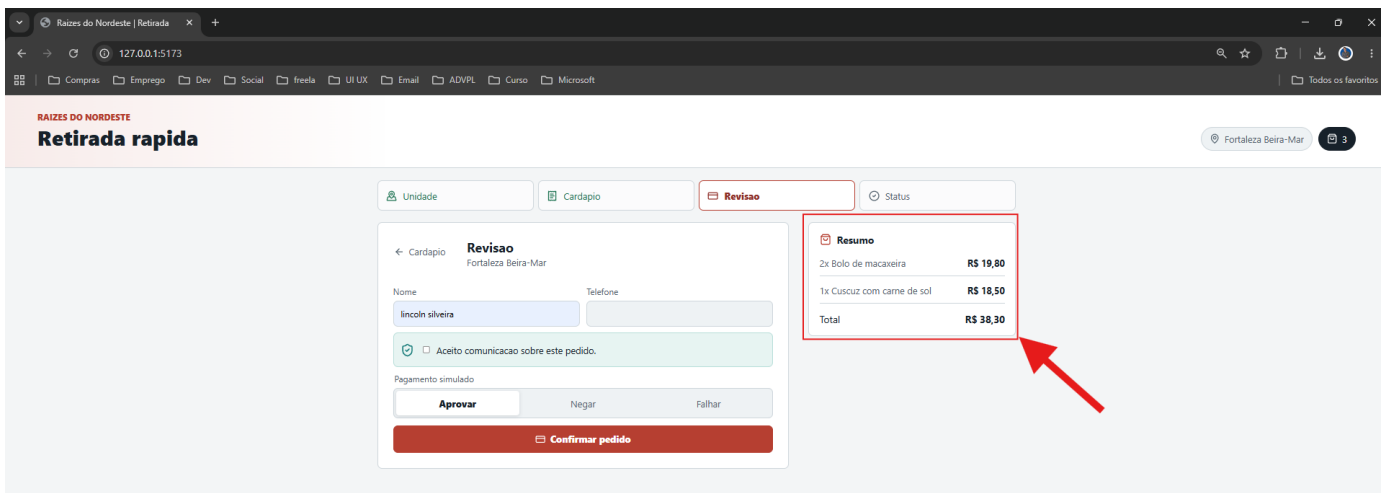
2 - Cardápio da unidade selecionada



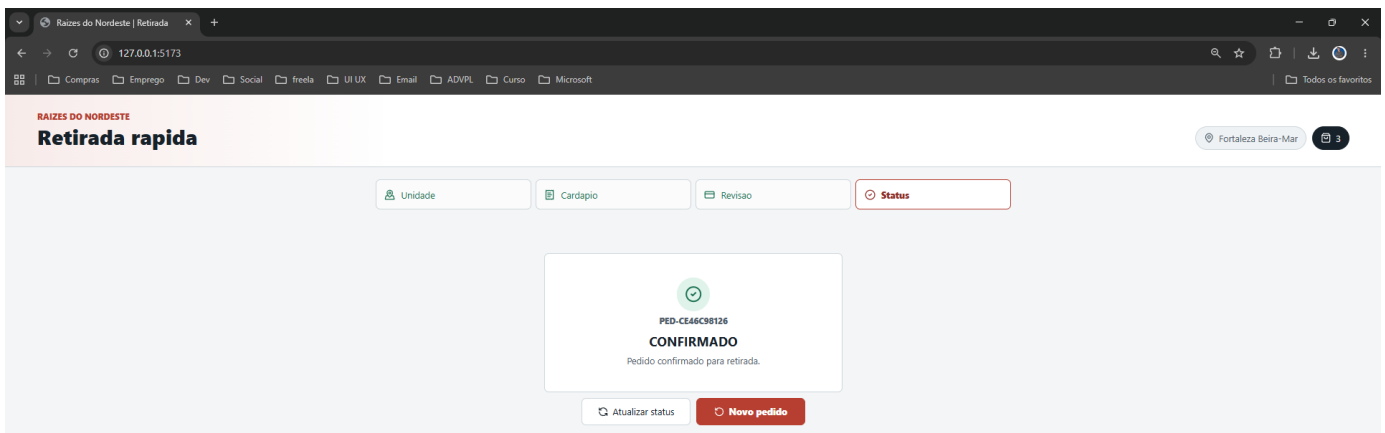
3 – Seleção de produtos disponíveis para unidade



4 – Revisão dos itens selecionados, quantidades e valor total antes da confirmação.



5 - Pedido confirmado e o status final apresentado ao usuário.



Mobile:**1 - Seleção de unidade:****2 - Cardápio da unidade selecionada:****3 – Revisão dos itens selecionados, quantidades e valor total antes da confirmação:**

11:43

Revisao Status

← Cardapio

Revisao
Fortaleza Beira-Mar

Nome
[Input Field]

Telefone
[Input Field]

☐ Aceito comunicacao sobre este pedido.

Pagamento simulado

Aprovar Negar Falhar

Confirmar pedido

Resumo

2x Bolo de macaxeira	R\$ 19,80
Total	R\$ 19,80

4 - Pedido confirmado e o status final apresentado ao usuário:

11:43

RAIZES DO NORDESTE

Retirada rapida

Fortaleza Beira-Mar 2

Unidade Cardapio

Revisao Status

☒

PED-AA09431AAC
CONFIRMADO
Pedido confirmado para retirada.

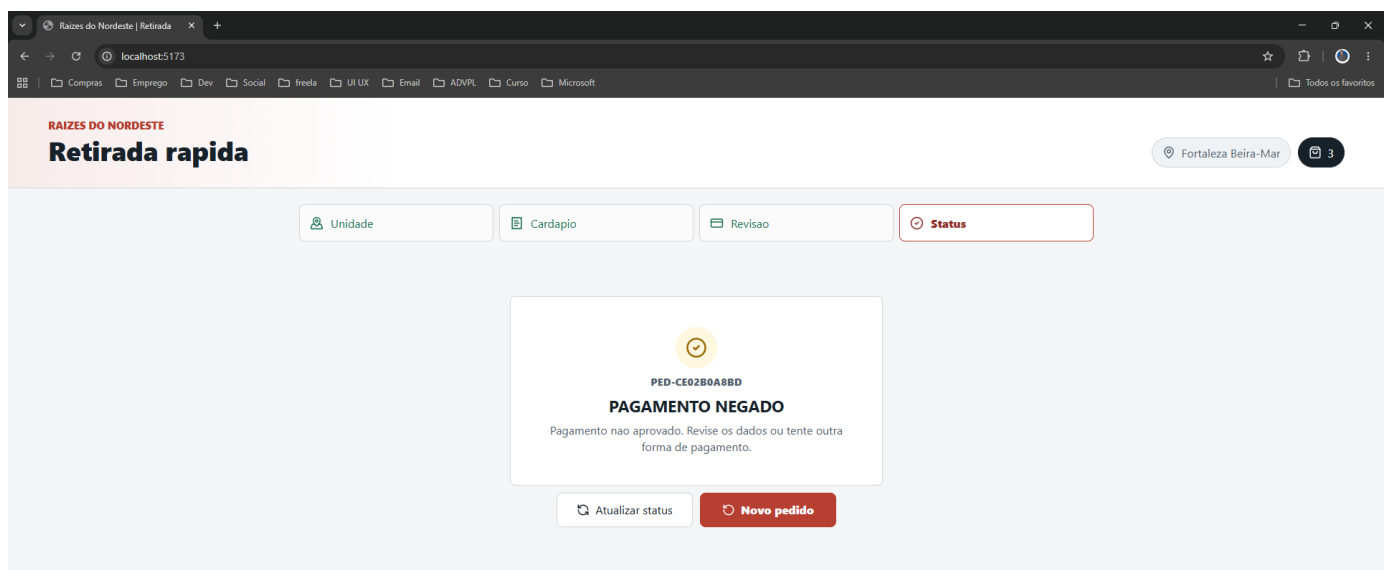
Atualizar status

Novo pedido

EV-006 - Pagamento negado

Cenário relacionado	CT-02 — Pagamento negado
Requisito relacionado	RF-09, RF-10, RF-12 / CA-08
Resultado esperado	Pedido válido não deve ser confirmado quando o pagamento simulado retornar NEGADO
Resultado obtido	O sistema exibiu mensagem de pagamento negado e manteve o pedido sem confirmação
Status	Aprovado

Tratamento do cenário de pagamento negado. Mesmo com um pedido válido, o sistema não confirma a retirada quando o serviço de pagamento simulado retorna negativa, exibindo uma mensagem compreensível ao usuário e mantendo o status coerente com o resultado do pagamento.

**EV-007 - Falha de pagamento com rastreo**

Cenário relacionado	CT-03 — Falha de pagamento
Requisito relacionado	RF-10, RF-13, RF-15 / RQ-08 / CA-09
Resultado esperado	O sistema deve tratar falha de pagamento sem quebrar a aplicação, exibir mensagem clara e registrar rastreo ou auditoria
Resultado obtido	Falha de pagamento tratada, pedido não confirmado e evento/log registrado para rastreabilidade
Status	Aprovado

Tratamento de falha no pagamento simulado. O cenário válida a resiliência do fluxo quando o serviço externo de pagamento apresenta erro, garantindo que o sistema não confirme indevidamente o pedido, informe o

usuário de forma clara e registre rastreio para análise posterior.

The screenshot shows the 'Retirada rápida' interface for 'Fortaleza Beira-Mar'. The top navigation bar includes 'RAÍZES DO NORDESTE' and 'Retirada rápida'. The location is set to 'Fortaleza Beira-Mar' with a cart icon showing 3 items. The main content area has tabs for 'Unidade', 'Cardapio', 'Revisao', and 'Status'. The 'Revisao' tab is active, displaying a 'PAGAMENTO INDISPONÍVEL' error message: 'Nao foi possível concluir o pagamento agora. Tente novamente em instantes. Rastreio: AUD-8A048CE683.' Below the error, the 'Revisao' section shows the user's name 'lincoln silveira' and a 'Confirmar pedido' button. A 'Resumo' section on the right lists items: '2x Bolo de macaxeira' for R\$ 19,80 and '1x Cuscuz com carne de sol' for R\$ 18,50, totaling R\$ 38,30.

EV-008 - Produto indisponível na unidade

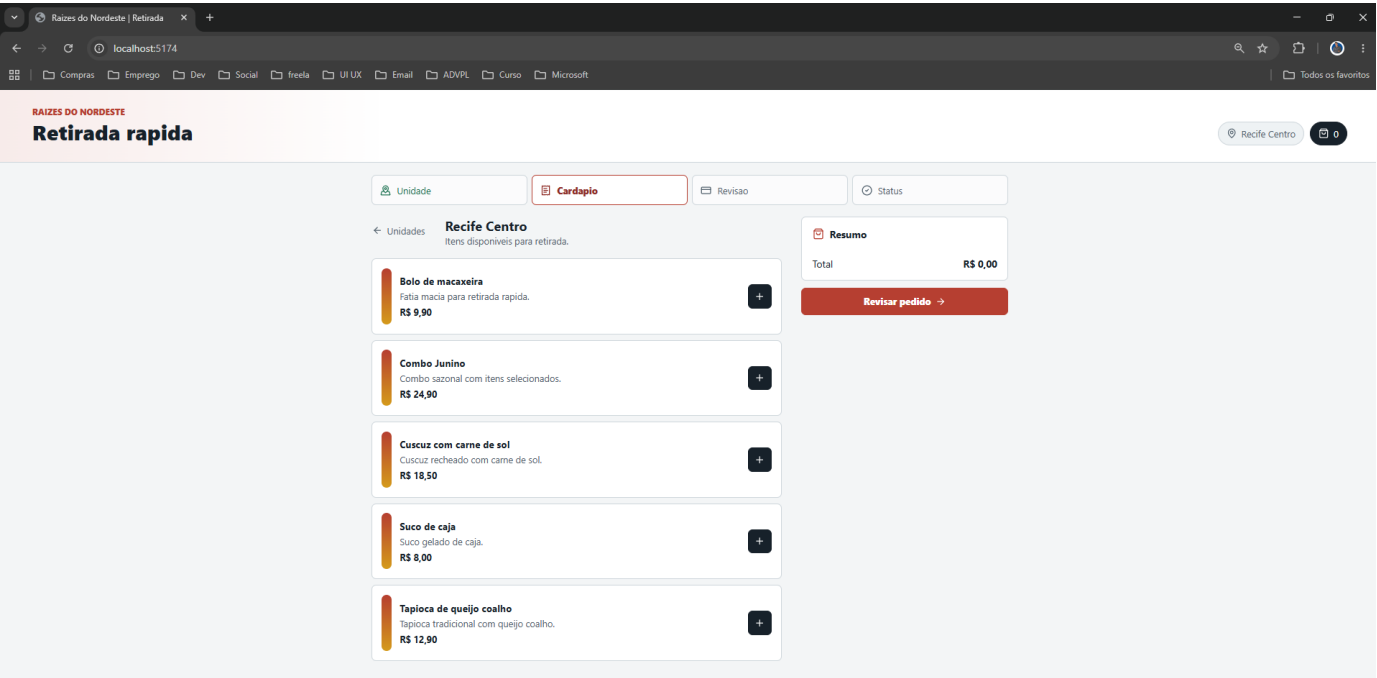
Cenário relacionado	CT-04 — Produto indisponível
Requisito relacionado	RF-02, RF-05 / RN-02 / CA-03 / RQ-04
Resultado esperado	O sistema deve rejeitar pedido contendo produto indisponível para a unidade selecionada
Resultado obtido	Pedido recusado com mensagem de erro informando indisponibilidade do produto
Status	Aprovado

Unidades:

Salvador Shopping

The screenshot shows the 'Retirada rápida' interface for 'Salvador Shopping'. The top navigation bar includes 'RAÍZES DO NORDESTE' and 'Retirada rápida'. The location is set to 'Salvador Shopping' with a cart icon showing 0 items. The main content area has tabs for 'Unidade', 'Cardapio', 'Revisao', and 'Status'. The 'Cardapio' tab is active, displaying a list of items: 'Bolo de macaxeira' for R\$ 9,90 and 'Tapioca de queijo coalho' for R\$ 12,90. A 'Resumo' section on the right shows a total of R\$ 0,00. A 'Revisar pedido' button is visible at the bottom right.

Recife Centro

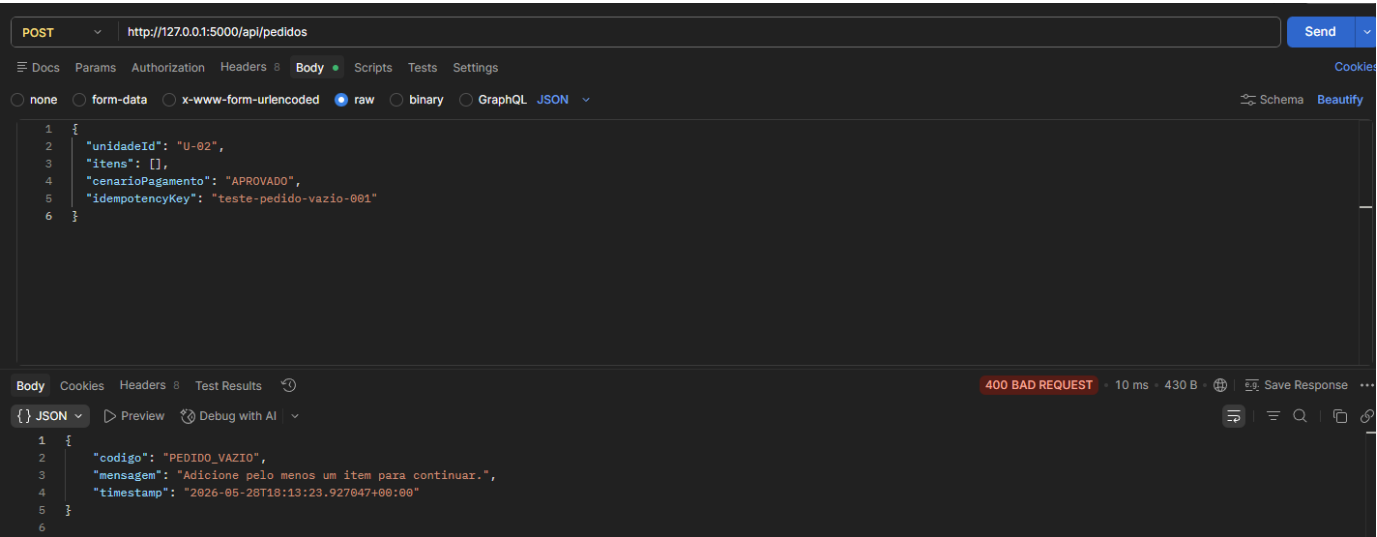


EV-009 - Validações negativas de pedido

Cenário relacionado	CT-05 — Pedido vazio / CT-06 — Quantidade inválida
Requisito relacionado	RF-06 / RN-03 / RN-04 / CA-04 / CA-05
Resultado esperado	O sistema deve rejeitar pedidos sem itens ou com quantidades inválidas
Resultado obtido	Pedidos inválidos foram recusados com retorno de erro e mensagem de validação.
Status	Aprovado

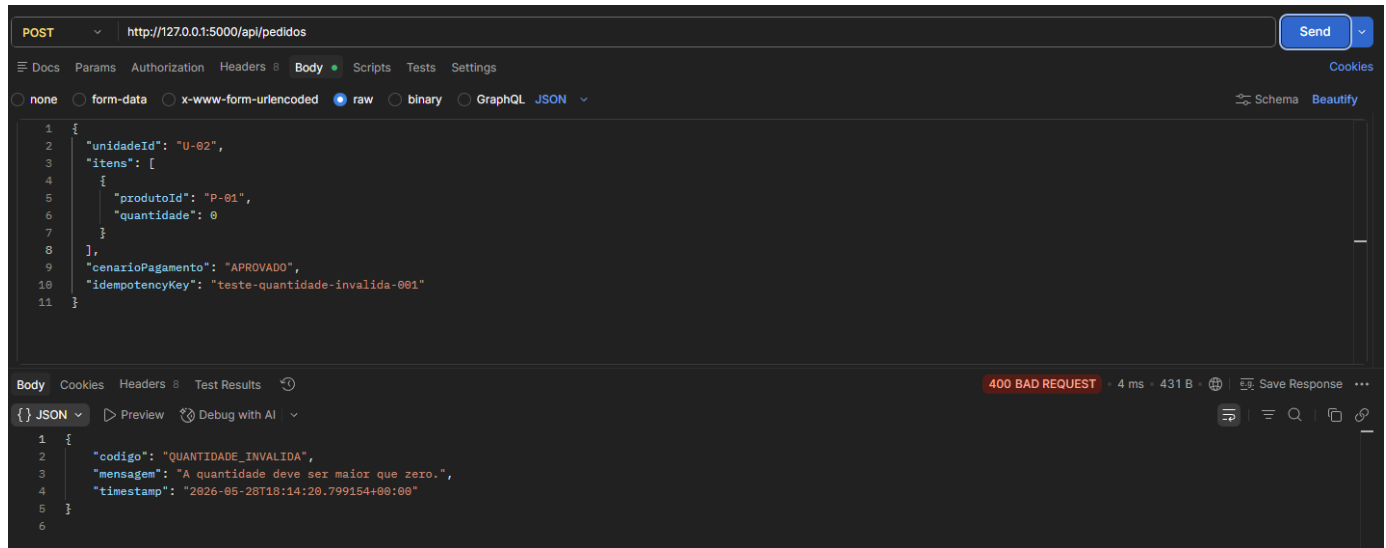
Pedido vazio:

Evidência de que o sistema rejeita a criação de pedido sem itens, impedindo a abertura de pedido inválido.



Quantidade inválida:

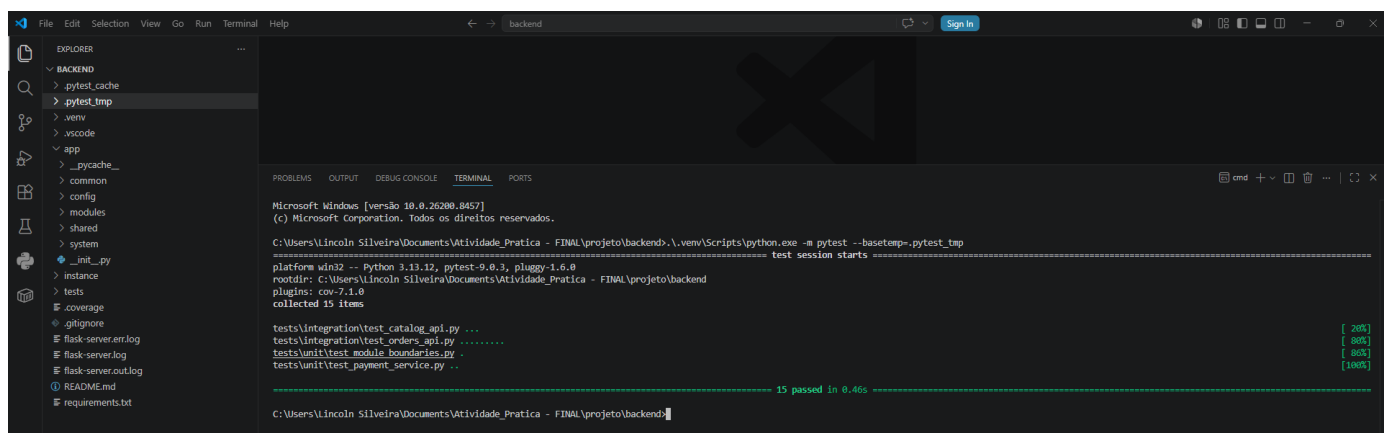
Evidência de que o sistema rejeita itens com quantidade zero ou negativa.



EV-010 - Testes unitários/backend

Cenário relacionado	Testes automatizados de backend
Requisito relacionado	RNF-10 — Manutenibilidade / RQ-03 — Confiabilidade / RF-06 / RF-09 a RF-13
Resultado esperado	Suíte de testes backend executada com sucesso, validando regras críticas e integrações da API
Resultado obtido	15 testes executados e aprovados com pytest
Status	Aprovado

Execução da suíte automatizada do backend com pytest. Foram executados testes de integração e testes unitários cobrindo catálogo, pedidos, fronteiras entre módulos e serviço de pagamento simulado.



EV-011 - Cobertura de testes

Cenário relacionado	Cobertura de testes automatizados
Requisito relacionado	RQ-03 — Confiabilidade / RNF-10 — Manutenibilidade
Resultado esperado	Cobertura mínima de 80% nas regras críticas do backend
Resultado obtido	Cobertura total de 95% no backend, com 15 testes aprovados

Status	Aprovado
--------	----------

Relatório de cobertura dos testes automatizados do backend. A meta definida para a entrega foi cobertura mínima de 80% nas regras críticas, e o resultado obtido foi de 95% de cobertura total.

```

===== tests coverage =====
coverage: platform win32, python 3.13.12-final-0

Name                                Stmts  Miss  Cover   Missing
-----
app\__init__.py                      32      0  100%
app\common\_init__.py                0      0  100%
app\common\enums.py                 11      0  100%
app\common\errors.py                 32      7    78%   24, 37-42, 46-53
app\config\_init__.py                0      0  100%
app\config\database.py               38      1    97%   152
app\config\settings.py                6      0  100%
app\modules\_init__.py               0      0  100%
app\modules\auditoria\_init__.py      0      0  100%
app\modules\auditoria\dto.py          8      0  100%
app\modules\auditoria\repository.py   13      2    85%   31-40
app\modules\auditoria\service.py      9      0  100%
app\modules\cardapio\_init__.py        0      0  100%
app\modules\cardapio\controller.py    7      0  100%
app\modules\cardapio\dto.py           17      0  100%
app\modules\cardapio\repository.py    13      0  100%
app\modules\cardapio\service.py       15      0  100%
app\modules\pagamento\_init__.py      0      0  100%
app\modules\pagamento\controller.py   9      3    67%   12-18
app\modules\pagamento\dto.py         12      5    58%   11-19
app\modules\pagamento\repository.py   8      0  100%
app\modules\pagamento\service.py     25      1    96%   42
app\modules\pedido\_init__.py          0      0  100%
app\modules\pedido\controller.py      11      0  100%
app\modules\pedido\dto.py             56      2    96%   57, 61
app\modules\pedido\repository.py       25      2    92%   87-95
app\modules\pedido\service.py         80      2    98%   140, 154
app\modules\privacidade\_init__.py     0      0  100%
app\modules\privacidade\dto.py         5      0  100%
app\modules\privacidade\repository.py  8      0  100%
app\modules\privacidade\service.py     9      0  100%
app\modules\unidade\_init__.py         0      0  100%
app\modules\unidade\controller.py      7      0  100%
app\modules\unidade\dto.py            9      0  100%
app\modules\unidade\repository.py     13      0  100%
app\modules\unidade\service.py        16      1    94%   19
app\shared\_init__.py                0      0  100%
app\shared\utils\_init__.py           0      0  100%
app\shared\utils\utils.py             3      0  100%
app\shared\utils\utils.py             3      0  100%
app\system\_init__.py                0      0  100%
app\system\controller.py              5      1    80%   9

TOTAL                                497      27    95%
15 passed in 0.74s

```

EV-012 - Testes frontend/build

Cenário relacionado	Testes e build da interface frontend
Requisito relacionado	RNF-04 — Usabilidade / RNF-05 — Responsividade / RNF-10 — Manutenibilidade
Resultado esperado	Testes do frontend executados com sucesso e build concluído sem erros
Resultado obtido	2 arquivos de teste e 3 testes aprovados no Vitest; build Vite/TypeScript concluído com sucesso em 1,61s
Status	Aprovado

Validação técnica do frontend. Foram executados os testes configurados para a interface e o processo de build da aplicação React com TypeScript, verificando se a aplicação compila corretamente e está apta para execução da jornada de pedido para retirada.

Testes frontend:

Evidência da suíte de testes frontend executada com sucesso.

```

C:\Users\Lincoln Silveira\Documents\Atividade_Pratica - FINAL\projeto\frontend>npm test

> raizes-qa-frontend@0.1.0 test
> vitest run

RUN v3.2.4 C:/Users/Lincoln Silveira/Documents/Atividade_Pratica - FINAL/projeto/frontend
✓ src/features/pedido/utils/cart.test.ts (2 tests) 2ms
✓ src/services/apiClient.test.ts (1 test) 5ms

Test Files  2 passed (2)
Tests       3 passed (3)
Start at    15:33:57
Duration   306ms (transform 55ms, setup 0ms, collect 67ms, tests 7ms, environment 0ms, prepare 201ms)

```

Build frontend:

Evidência da compilação da aplicação React/TypeScript concluída sem erros críticos.

```

C:\Users\Lincoln Silveira\Documents\Atividade_Pratica - FINAL\projeto\frontend>npm run build

> raizes-qa-frontend@0.1.0 build
> tsc --noEmit && vite build

vite v6.4.2 building for production...
✓ 1651 modules transformed.
dist/index.html                0.42 kB | gzip: 0.28 kB
dist/assets/index-DWdCb-qq.css  8.70 kB | gzip: 2.40 kB
dist/assets/index-CQdWdKl.js    213.90 kB | gzip: 66.60 kB
✓ built in 1.61s

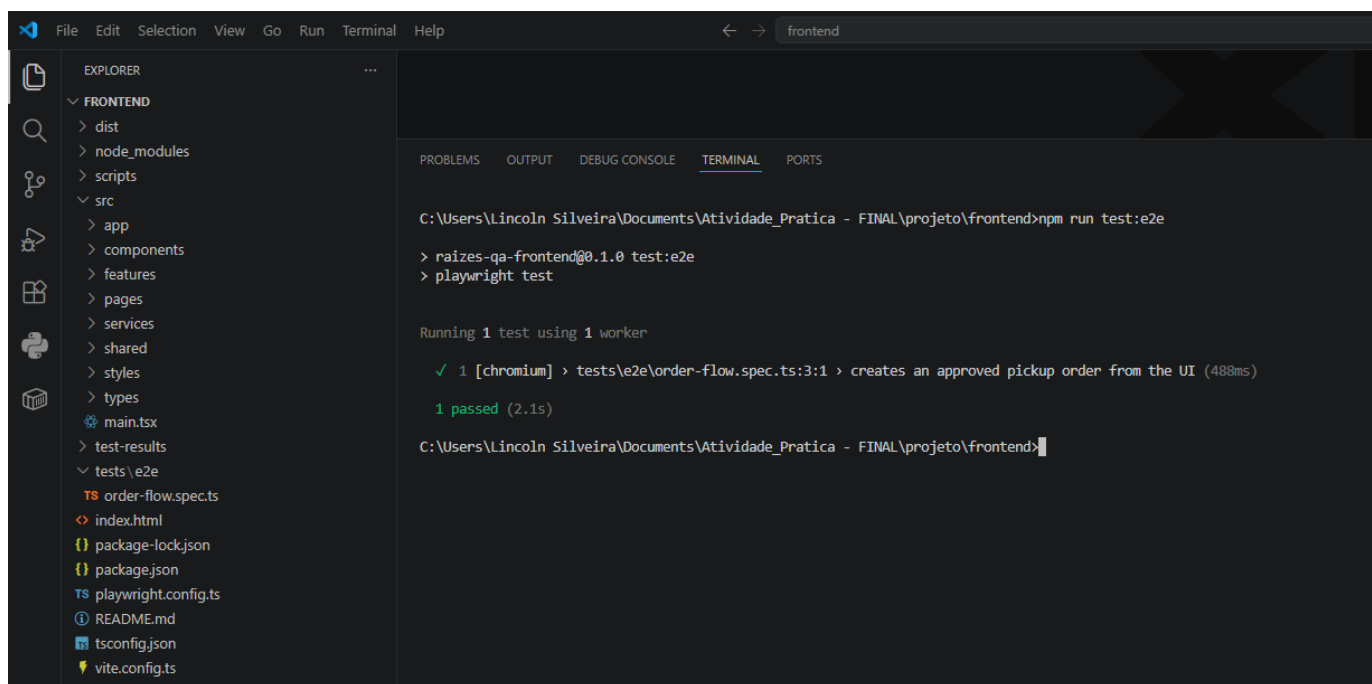
C:\Users\Lincoln Silveira\Documents\Atividade_Pratica - FINAL\projeto\frontend>

```

EV-013 - Teste E2E

Cenário relacionado	CT-01 — Pedido aprovado ponta a ponta
Requisito relacionado	RF-01, RF-02, RF-03, RF-07, RF-09, RF-10, RF-11, RF-14 / RQ-05
Resultado esperado	O teste E2E deve validar o fluxo completo de pedido, desde a seleção da unidade até o status final
Resultado obtido	Teste E2E executado com sucesso pelo Playwright
Status	Aprovado

Execução de teste ponta a ponta da aplicação. O objetivo foi validar o comportamento integrado da jornada principal de pedido para retirada rápida, contemplando seleção de unidade, cardápio, carrinho, confirmação, pagamento simulado e status final.



EV-014 - Responsividade mobile

Cenário relacionado	CT-11 — Fluxo mobile
Requisito relacionado	RNF-05 — Responsividade / RQ-06 — Responsividade
Resultado esperado	O fluxo principal deve funcionar em viewports mobile sem quebra crítica de layout
Resultado obtido	As telas principais do fluxo foram exibidas corretamente em resolução mobile, mantendo legibilidade e navegação
Status	Aprovado

Validação de responsividade mobile da jornada de pedido para retirada rápida. Foram verificados os principais pontos do fluxo em viewport reduzido, incluindo seleção de unidade, cardápio, revisão do pedido, privacidade e status final, com o objetivo de confirmar legibilidade, organização visual e ausência de quebras críticas na interface.

Android comum (360 x 800):

Seleção de unidade:



Cardápio:



Carrinho/revisão e LGPD/privacidade:

Nome

Telefone

Privacidade e uso dos dados

Usamos nome e telefone apenas para identificar o pedido e comunicar atualizações sobre a retirada. O pagamento é simulado e não solicita dados financeiros reais.

☐ Autorizo o uso do meu contato apenas para comunicação sobre este pedido.

Pagamento simulado

Aprovar

Negar

Falhar

Confirmar pedido

Resumo

2x Bolo de macaxeira

R\$ 19,80

1x Tapioca de queijo coalho

R\$ 12,90

Total

R\$ 32,70

Status final:

RAIZES DO NORDESTE

Retirada rapida

Salvador Shopping

2

Unidade

Cardapio

Revisao

Status

PED-BEDF103880

CONFIRMADO

Pedido confirmado para retirada.

Atualizar status

Novo pedido

EV-015 - LGPD e privacidade

Cenário relacionado	CT-12 — Consentimento aplicável
Requisito relacionado	RF-17, RF-18 / RNF-07 / RNF-08 / RQ-07 / CA-12
Resultado esperado	A interface deve informar a finalidade do uso dos dados, apresentar consentimento quando aplicável e não solicitar dados financeiros reais
Resultado obtido	O fluxo exibiu aviso de privacidade, finalidade de uso de nome e telefone, consentimento para comunicação e informação de pagamento simulado sem dados financeiros reais
Status	Aprovado

Aplicação de princípios de LGPD e privacidade no fluxo de pedido. A interface informa a finalidade do uso de nome e telefone, apresenta consentimento específico para comunicação sobre o pedido e esclarece que o pagamento é simulado, sem solicitação de dados financeiros reais.

EV-016 - Desempenho/carga

Cenário relacionado	CT-09 — Cardápio dentro da meta / CT-10 — Pedido dentro da meta
Requisito relacionado	RNF-01, RNF-02 / RQ-01, RQ-02
Resultado esperado	Endpoints principais devem responder com P95 menor ou igual a 2 segundos e taxa de erro inferior a 1%
Resultado obtido	Teste com k6 executado com 20 usuários virtuais por 30 segundos, 2400 checks realizados, 100% de sucesso, 0% de falhas e P95 de 7,91ms
Status	Aprovado

Execução de teste básico de desempenho com k6. O objetivo foi validar o comportamento dos endpoints de unidades e cardápio sob carga controlada, considerando como meta tempo de resposta P95 inferior a 2 segundos e taxa de erro inferior a 1%.

```

execution: local
script: k6-carga-basica.js
output: -

scenarios: (100.00%) 1 cenário, 20 max VUs, 1m0s max duration (incl. graceful stop):
* default: 20 looping VUs for 30s (gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<2000' p(95)=7.91ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 2400    79.46599/s
checks_succeeded...: 100.00% 2400 out of 2400
checks_failed.....: 0.00%   0 out of 2400

✓ GET /api/unidades status 200
✓ GET /api/unidades tempo < 2s
✓ GET /api/unidades/U-02/cardapio status 200
✓ GET /api/unidades/U-02/cardapio tempo < 2s

HTTP
http_req_duration.....: avg=2.76ms min=469.3µs med=2.07ms max=22.43ms p(90)=3.71ms p(95)=7.91ms
{ expected_response:true }...: avg=2.76ms min=469.3µs med=2.07ms max=22.43ms p(90)=3.71ms p(95)=7.91ms
http_req_failed.....: 0.00% 0 out of 1200
http_reqs.....: 1200    39.732995/s

EXECUTION
iteration_duration.....: avg=1s    min=1s    med=1s    max=1.04s p(90)=1s p(95)=1.02s
iterations.....: 600    19.866497/s
vus.....: 20    min=20    max=20
vus_max.....: 20    min=20    max=20

NETWORK
data_received.....: 685 kB 23 kB/s
data_sent.....: 187 kB 3.5 kB/s

running (0m30.2s), 00/20 VUs, 600 complete and 0 interrupted iterations
default ✓ [=====] 20 VUs 30s

```

EV-017 - Checklist UAT/usabilidade

Cenário relacionado	Teste de aceitação e usabilidade da jornada
Requisito relacionado	RNF-04 — Usabilidade / RNF-05 — Responsividade / RQ-05 — Usabilidade / RQ-06 — Responsividade
Resultado esperado	A jornada deve ser compreensível, navegável e permitir que o usuário conclua o pedido sem esforço excessivo
Resultado obtido	Checklist UAT/usabilidade preenchido com aprovação dos principais critérios de clareza, navegação, status, mensagens e fluxo mobile
Status	Aprovado

Checklist de aceitação e usabilidade aplicado à jornada de pedido para retirada rápida. A validação avaliou clareza da unidade selecionada, apresentação do cardápio, revisão do pedido, mensagens de erro, status final e comportamento em interface mobile.

Item	Status	Observação
Usuário entende qual unidade está selecionada	Aprovado	A unidade selecionada aparece no topo e no fluxo de revisão.
Cardápio apresenta itens de forma clara	Aprovado	Os produtos são listados com nome, descrição, preço e ação de adicionar.
Carrinho mostra itens, quantidades e total	Aprovado	O resumo apresenta os itens selecionados, quantidade e valor total.
Mensagem de erro é compreensível	Aprovado	Cenários negativos apresentam retorno controlado e mensagens claras.
Status final do pedido é fácil de identificar	Aprovado	O sistema exibe o resultado final do pedido após a confirmação.
Fluxo mobile não exige esforço excessivo	Aprovado	As etapas principais permanecem legíveis e navegáveis em viewport mobile.

6. Conclusão

O projeto demonstra que Garantia da Qualidade de Software é uma prática estratégica para sistemas integrados de redes de franquias. A qualidade não foi tratada apenas como execução de testes após o desenvolvimento, mas como uma trilha de decisão que começa em riscos e requisitos, passa por arquitetura e critérios de aceitação, e termina em evidências e métricas.

A escolha da feature de pedido para retirada rápida com pagamento simulado permitiu validar uma jornada crítica do cliente sem implementar o sistema completo. Essa abordagem respeita o escopo acadêmico e, ao mesmo tempo, aproxima a entrega de um caso real de desenvolvimento, pois integra backend, frontend, regra de negócio, testes e documentação.

6.1 Lições aprendidas

Lição	Descrição
Requisitos precisam ser mensuráveis	Frases genéricas como "rápido" ou "seguro" foram transformadas em metas e evidências.
Arquitetura influencia qualidade	A separação por domínio melhora testabilidade, manutenção e rastreabilidade.
Testes precisam de contexto	Cada teste foi ligado a risco, requisito, critério e evidência.
LGPD deve aparecer no fluxo	Privacidade não fica apenas em texto jurídico; precisa ser clara na interface e nos logs.
Evidência é parte da entrega	Prints, logs e relatórios tornam a validação auditável.

6.2 Desafios e pontos de atenção

Os principais desafios do projeto foram delimitar o escopo, evitar implementação excessiva, organizar responsabilidades por domínio e transformar critérios de qualidade em validações práticas. Como próximos passos, recomenda-se ampliar testes E2E negativos, consolidar teste de carga, melhorar evidências de responsividade e manter a documentação atualizada conforme o código evoluir.

6.3 Evoluções futuras

Evolução	Justificativa
Fidelização	Consumir pedidos confirmados e aplicar regras com consentimento LGPD.
Indicadores gerenciais	Apoiar matriz com vendas, produtos mais consumidos, falhas e SLA.

Evolução	Justificativa
Autenticação e perfis	Separar cliente, unidade, matriz e auditoria.
Estoque real	Evitar pedido de item indisponível em tempo real.
Observabilidade avançada	Métricas, logs estruturados e alertas por domínio.

7. Referências

- UNINTER. Orientações e Estudo de Caso - Rede Raízes do Nordeste. Material da atividade prática.
- UNINTER. Roteiro da Trilha de Qualidade de Software. Material da atividade prática.
- Documentação interna do projeto: visão, requisitos, arquitetura, API, QA e execução.
- SOMMERVILLE, Ian. Engenharia de Software. Referência conceitual para requisitos, qualidade e testes.
- PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. Referência conceitual para qualidade de software.
- Lei nº 13.709/2018 - Lei Geral de Proteção de Dados Pessoais (LGPD).
- OWASP Foundation. Materiais de referência para segurança de aplicações web.
- Documentações oficiais das ferramentas previstas: Flask, pytest, React, Vite, Vitest, Playwright, k6/JMeter.

Declaração de uso de IA

Foi utilizada ferramenta de IA apenas como apoio para revisão textual, organização do documento e verificação de aderência ao roteiro. A definição do recorte, implementação, testes, evidências, prints e decisões técnicas foram conduzidas pelo aluno.

Ferramenta utilizada

ChatGPT, da OpenAI.

Finalidade do uso

- Revisão ortográfica e gramatical do documento.
- Apoio na organização das seções do trabalho.
- Sugestões de melhoria de clareza e padronização textual.
- Apoio na verificação de coerência entre o documento e o roteiro da Trilha de Qualidade de Software.
- Sugestões gerais para explicitar Plano de Qualidade, Plano de Testes, Cronograma de Validação, Papéis e Responsabilidades, métricas e evidências.

Exemplos de comandos/prompts utilizados

- “Revise se o documento está coerente com o roteiro da Trilha de Qualidade de Software.”
- “Sugira melhorias de organização textual para o plano de qualidade.”
- “Verifique se os itens obrigatórios do roteiro aparecem de forma explícita no documento.”
- “Melhore a clareza da seção de métricas e indicadores sem alterar o sentido técnico.”

Trechos aproveitados

Foram aproveitadas sugestões de revisão textual, organização de seções, melhoria de redação e explicitação de itens exigidos pelo roteiro.